Q11. Logistic Regression Classification

Original Aim: To classify Iris flowers into their Species categories using Logistic Regression. Use the four numerical attributes as input features and Species as the target. Split the dataset into training and testing sets, train the classifier, make predictions, and report accuracy and the confusion matrix.

Aim

Classify Species from the four numerical attributes and report test accuracy and a labelled confusion matrix.

Requirements

Python, Pandas, NumPy, Matplotlib and scikit-learn. References: [D11], [D12].

Essential theory

Classification predicts categories; regression predicts numerical quantities. Logistic Regression uses linear scores to model class probabilities and choose a label; despite its name it is a classifier here. Accuracy=correct predictions/total test predictions. In the confusion matrix below, rows are actual classes and columns are predicted classes. Diagonal entries are correct; off-diagonal entries are errors.

Step-by-step procedure

1. Select the four numerical features as X and Species as y.
2. Split into 80% training/20% test with random_state=42 and stratify=y, preserving equal class proportions.
3. Build a Pipeline containing StandardScaler then LogisticRegression with lbfgs and max_iter=1000.
4. Fit the Pipeline on training data only, then predict X_test.
5. Calculate accuracy and confusion_matrix using an explicit species order.
6. Display predictions, label rows/columns and generate the saved confusion-matrix figure.

Complete Python program

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)

X = df[features]
y = df["Species"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model = Pipeline([
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(solver="lbfgs", max_iter=1000))
])
model.fit(X_train, y_train)
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
cm = confusion_matrix(y_test, predictions, labels=species)
cm_table = pd.DataFrame(cm, index=species, columns=species)
comparison = pd.DataFrame({"Actual": y_test, "Predicted": predictions})
print("Train/test shapes:", X_train.shape, X_test.shape)
print(f"Accuracy: {accuracy:.6f}")
print("Rows = actual; columns = predicted; order:", species)
print(cm_table.to_string())
print("ALL TEST PREDICTIONS (original row index)")
print(comparison.to_string())
fig, ax = plt.subplots(figsize=(6, 4.5))
ax.imshow(cm, cmap="Blues", vmin=0)
short_labels = [name.replace("Iris-", "") for name in species]
ax.set_xticks(range(3), short_labels, rotation=20)
ax.set_yticks(range(3), short_labels)
for row, col in np.ndindex(cm.shape):
    ax.text(col, row, str(cm[row, col]), ha="center", va="center",
            color="white" if cm[row, col] > cm.max() / 2 else "black")
ax.set_xlabel("Predicted species")
ax.set_ylabel("Actual species")
ax.set_title("Logistic regression: held-out confusion matrix")
fig.tight_layout()
fig.savefig("q11_confusion.png", dpi=180)
plt.show()
```

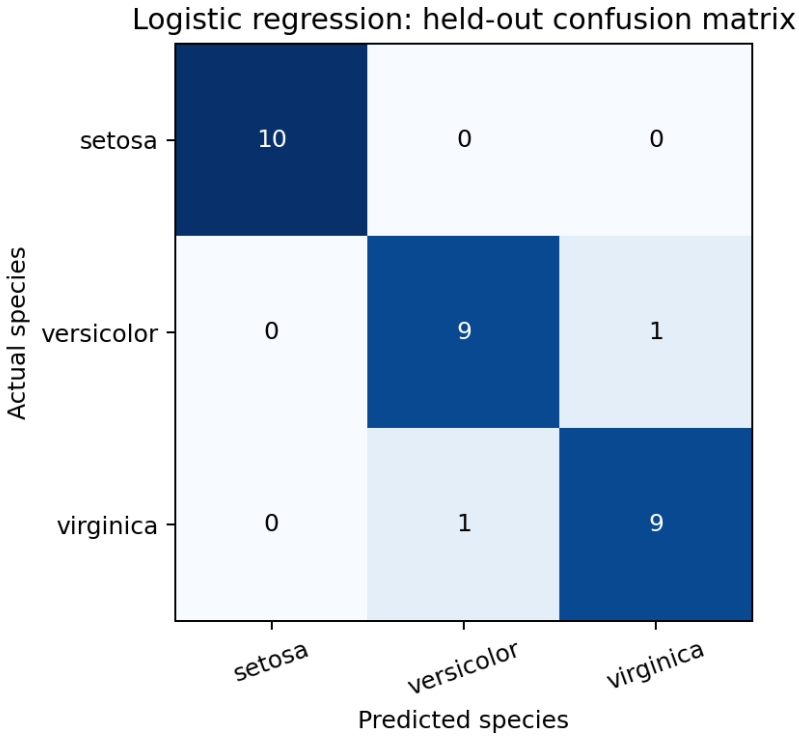
Code explanation

- `stratify=y` gives 40 training and 10 test flowers per class for this balanced dataset.
- The Pipeline fits the scaler on X_train only; test rows are transformed using those learned means/scales. It avoids leaking test distribution information into fitting.
- `max_iter=1000` provides a practical convergence budget; it is not a guarantee of convergence on arbitrary data. No convergence warning occurred in this run.

- `labels=species` fixes matrix order as Iris-setosa, Iris-versicolor, Iris-virginica, independent of which predictions appear.
- Matplotlib renders the actual computed matrix; it is not a hand-drawn illustrative result. Numbers are annotated and axes explicitly show actual versus predicted.

Verified output and figure

```
Train/test shapes: (120, 4) (30, 4)
Accuracy: 0.933333
Rows = actual; columns = predicted; order: ['Iris-setosa', 'Iris-versicolor',
  'Iris-virginica']
                 Iris-setosa  Iris-versicolor  Iris-virginica
Iris-setosa               10                0               0
Iris-versicolor            0                9               1
Iris-virginica             0                1               9
TEST PREDICTION EXCERPT: first five rows plus both errors
Actual        Predicted
38       Iris-setosa      Iris-setosa
127   Iris-virginica   Iris-virginica
57   Iris-versicolor  Iris-versicolor
93   Iris-versicolor  Iris-versicolor
42       Iris-setosa      Iris-setosa
134   Iris-virginica  Iris-versicolor
77   Iris-versicolor   Iris-virginica
```



Observation

The program prints all test predictions; the PDF output excerpt shows the first five and both errors. The model correctly classifies 28 of 30 test flowers, giving 93.33% accuracy. All 10 setosa examples are correct. One actual versicolor is predicted as virginica (row 77); one actual virginica is predicted as versicolor (row 134). The two non-setosa classes are harder to separate in this split. A single small held-out split does not establish the accuracy for every future dataset.

Result

A scaled multiclass Logistic Regression classifier was trained and evaluated on held-out data. Its accuracy and confusion matrix agree: diagonal sum 10+9+9=28, total=30.

Common errors and troubleshooting

- ConvergenceWarning: check numeric data/scaling; increase the iteration budget if needed and rerun. Do not simply hide the warning.
- Unknown or missing target values: inspect y and labels before splitting; this practical uses the complete original target, not Q3's altered copy.
- Matrix interpreted backwards: keep rows=actual and columns=predicted and state label order.
- Score computed on training rows: pass y_test and predictions from X_test to the metric.

Viva questions and answers

- Why is Logistic Regression a classification model? It models class probabilities and predicts discrete labels.
- What does stratification preserve? Approximate class proportions in the train and test partitions.
- What is an off-diagonal matrix entry? A misclassification from its actual row class to its predicted column class.
- Is high training accuracy sufficient? No; held-out evaluation is needed to assess generalization.